

## ◆ Git Advanced Questions & Answers

### 1. What is the difference between `git merge` and `git rebase`?

**Answer:**

**Merge:** Combines changes from one branch into another by creating a **new merge commit**. Keeps history as it is (non-linear).

Example:

```
git checkout main
git merge feature
```

→ Creates a new commit that merges `feature` into `main`.

**Rebase:** Moves or reapplies commits from one branch **on top of another branch's HEAD**, creating a **linear history**.

Example:

```
git checkout feature
git rebase main
```

→ Rewrites commits of `feature` onto `main`.

**Key difference:**

- Merge = preserves history, easy for collaboration.
- Rebase = cleaner history, but rewrites commits (dangerous in shared branches).

---

### 2. When should you use `rebase` over `merge`?

**Answer:**

- Use **merge** when:
  - Working on a shared branch with multiple developers.
  - You want to keep the exact history of merges.

- In long-lived branches where you don't want to rewrite history.
  - Use **rebase** when:
    - You want a **clean, linear history** (e.g., before merging the feature into `main`).
    - You are the **only one working on the branch**, so rewriting commits won't affect others.
    - You want to avoid unnecessary merge commits in the history.
- 

### 3. What are the risks of using `git rebase`?

Answer:

- **Rewriting history:** Rebasing changes commit hashes, so if the branch is already pushed/shared, it may cause conflicts for others.
  - **Data loss:** If not used carefully, commits can be lost (especially with `git rebase --onto` or `git rebase --interactive`).
  - **Harder debugging:** Merge commits show the context of when branches diverged, which rebasing removes.
- 

### 4. How do you resolve conflicts during a `git rebase`?

Answer:

1. Start rebase:

```
git rebase main
```

2. If conflict occurs:

- Git pauses rebase.
- Conflicted files will be marked (`<<<<<<, =====, >>>>>>`).

3. Fix conflicts manually in the files.

4. Mark file as resolved:

```
git add <file>
```

5. Continue rebase:

```
git rebase --continue
```

6. If stuck, you can abort:

```
git rebase --abort
```

---

## 5. What is an interactive rebase and why is it useful?

**Answer:**

Command:

```
git rebase -i HEAD~5
```

- **Interactive rebase** lets you edit commit history:
    - Reorder commits
    - Squash multiple commits into one
    - Edit commit messages
    - Drop unwanted commits
  - **Use cases:**
    - Cleaning up history before pushing a feature branch.
    - Combining “WIP” commits into meaningful commits.
    - Splitting a large commit into smaller logical commits.
- 

## 6. How do you handle conflicts in `git merge`?

**Answer:**

1. Run:

```
git merge feature
```

2. If conflict happens, Git stops the merge.

3. Check conflicted files:

```
git status
```

4. Manually resolve conflicts.
5. Mark resolved:

```
git add <file>
```

6. Complete merge:

```
git commit
```

---

## 7. How do you resolve a conflict where the same line is changed in both branches?

Answer:

- Git marks conflict with:

```
<<<<<< HEAD
code from current branch
=====
code from merging branch
>>>>>> feature
```

- Options:
  - Keep only **one version**.
  - Combine changes manually.
  - Use merge tools:

```
git mergetool
```

---

## 8. What is the difference between `git rebase --continue`, `--skip`, and `--abort`?

Answer:

- `--continue`: After fixing conflicts, continue the rebase.

- `--skip`: Skip the problematic commit and continue rebasing the rest.
  - `--abort`: Cancel the rebase and return branch to original state.
- 

## 9. Can you rebase a branch that has already been pushed to remote?

Answer:

- **Not recommended** if others are using the branch → it will rewrite commit history and cause issues.
- If you must, use **force push** after rebase:

```
git push --force
```

- Safe if branch is personal (not shared).
- 

## 10. How do you squash multiple commits into one using rebase?

Answer:

1. Run interactive rebase:

```
git rebase -i HEAD~3
```

2. In editor, change `pick` to `squash (s)` for commits you want to merge.
  3. Save and edit commit message.
  4. Final result → single commit instead of multiple.
- 

## 11. What is the difference between `git pull --rebase` and `git pull`?

Answer:

- `git pull`: Fetches remote branch and merges → creates merge commit.
  - `git pull --rebase`: Fetches remote branch and rebases your local commits on top → linear history, no merge commit.
-

**12. If you are in the middle of a merge/rebase and want to start fresh, what do you do?**

**Answer:**

For merge:

```
git merge --abort
```

For rebase:

```
git rebase --abort
```

This restores branch to pre-merge/rebase state.